Multi-Agent Reinforcement Learning for Derivative Model Calibration

Hadrian Fratarcangeli Benjamin Nicholson
Stony Brook University Stony Brook University

May 11, 2026

Abstract

Calibrating option pricing models to market data is a central problem in quantitative finance. The problem requires the identification of a volatility process that reproduces observed option prices across a wide range of strikes and maturities. Option prices deviate from any ground truth price, which suggests varying models and beliefs of the market's volatility - effectively breaking traditional frameworks such as Black Scholes. The resulting implied volatility surface exhibits a systematic structure contrary to the Black-Scholes assumptions, the volatility smile. The volatility surface structure highlights the inadequacy of constant-volatility models, motivating the need for more flexible dynamics.

This paper aims to replicate the results of Vadori (2022), who proposed a multi-agent reinforcement learning (MARL) framework for calibrating derivative pricing models. The MARL framework formulates calibration as an inverse problem: given a set of market option prices, recover a continuous-time diffusion model for the underlying asset whose prices match observed market prices. Vanilla options provide limited information about the joint dynamics of the asset across time, rendering the problem underdetermined. This challenge is amplified when considering path-dependent or exotic derivatives. Vadori (2022) focuses on the reconstruction of the implied volatility surface as the primary calibration target, leveraging its one-to-one correspondence with option prices under Black–Scholes. The objective is to identify a volatility model that, when used to simulate asset paths and price derivatives, reproduces the surface of market-implied volatilities.

1 Introduction

The calibration problem can be characterized as, given a set of options O with market prices P_{mkt}, we want to find the volatility process σ_t such that:

$$P_\sigma = P_{mkt} \text{ under some stochastic continuous diffusion model: } dS_t = \mu S_t dt + \sigma S_t dW_t$$

The price dynamics of an underlying asset can be modeled via a continuous diffusion process which is used Monte Carlo Simulation to determine the future potential price points. The expected payoff uses these potential price points across strikes and expiry dates to model an associated volatility surface which represents the assumed volatility process during the specific time period. This surface / smile can vary drastically depending on the underlying assumptions of the stochastic diffusion process.

1.1 The Black-Scholes Model

The Black-Scholes framework assumes that the underlying asset price S_t follows a Geometric Brownian Motion (GBM), characterized by the following stochastic differential equation (SDE):

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

where μ is the constant drift, σ is the constant volatility, and W_t is a standard Wiener process. This SDE implies that the log-returns of the asset are normally distributed:

$$\ln\left(\frac{S_t}{S_0}\right) \sim \mathcal{N}\left((\mu - \tfrac{1}{2}\sigma^2)t, \sigma^2 t\right)$$

Under these assumptions, using numerical solvers such as Brentq to map market prices to implied volatility it would result in a flat volatility surface. This flat volatility surface is the result of the assumed possible prices at the end of period following a log-normal distribution. In real-world financial markets, empirical observations consistently reveal stylized facts such as volatility clustering, "fat tails" (excess kurtosis) and regime shifting dynamics. These phenomena violate the independent and identically distributed (IID) normal assumption inherent in the GBM model. Consequently, the Black-Scholes model is treated as a benchmark for quoting prices rather than the ground truth. If the market truly followed this idealized environment, there would only ever be a single P_{mkt} under the constant σ for every option O regardless of its strike or expiry.

1.2 The Local Volatility (Dupire) Model

The local volatility model, introduced by Dupire (1994), assumes that the underlying asset follows a diffusion process:

$$dS_t = \mu S_t dt + \sigma(S_t, t) S_t dW_t$$

where $\sigma(S_t, t)$ is a non-parametric deterministic function. This local volatility model allows for our volatility to dynamically evolve which appropriately accounts for shifting market regimes in financial markets. The Dupire local volatility model works very well for liquid and defined option markets however suffer in exotic options that require path dependencies and well defined Greeks. This local volatility model can be achieved through the MARL framework. By utilizing the state space to be $x_t^i = (\ln S_t^i, t/T)$ we essentially have a framework for deriving the success of the Dupire model without depending on extensive market information.

1.3 Multi-Agent Reinforcement Learning

Advancements in computing power alongside the resurgence of neural networks and Reinforcement Learning (RL) has seen "Deep Reinforcement Learning (DRL)" become popular across many domains. The universal function approximation of the neural network alongside the sequential decision making in RL's Markov Decision Processes (MDP) has become ideal for many applications in the financial industry. Vadori (2022) as part of JP Morgan's overall move towards deep hedging introduced option calibration models via Multi-Agent DRL (MARL).

MARL is the extension of RL through the generalization of the singular MDP to multiple interacting MDPs. This extension has resulted in the intersection of game theory and reinforcement learning resulting in stochastic games. Game theory defines two main types of games - a cooperative and non cooperative game. These two games differ in how the agents within the game share information and work towards goals. In the case of MARL for calibration of options prices there is a shared goal of fitting implied volatility of the simulation to the market implied volatility - therefore our agents will be cooperative in this game.

Agents interactions are typically modeled with a one-to-one correspondence however this has a complexity of $O(N^2)$ which becomes infeasible for the calibration of options. Therefore we assume an agent views the collection of all other volatilities as its interaction rather than each agent individually. Resulting in a high agent optimization problem thorugh the lens of a single optimization for each agent under the same policy network.

Geometric Brownian Motion is the generalized SDE for pricing assets but for the MARL setup it requires specific attributes. We need to first assume that the mean growth rate is the risk free rate which is a general assumption made under the risk neutral measure to achieve a discounted martingale. However Vadori highlights that setting r = 0 is needed to completely remove the drift component, consequently reducing the price dynamic complexity to:

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

. Forward prices are used in the calibration as without any trend in the model dynamics it would assume an undiscounted martingale under the risk-neutral measure. This is an incorrect assumption of the market and would result in an upper skewed final trajectory histogram to meet the growth rate of of the underlying asset over that period. As a result we use the forward price of the underlying asset using the put-call parity to derive r.

The put-call parity allows us to find the forward price of an underlying asset given a certain time til expiry (tau). The difference in a European call and European put for same strikes and tau have the following relationship:

$$C + Ke^{-rT} = P + S_0$$
$$C - P = S_0 - Ke^{-rT}$$

By using market call and put contracts it is possible to derive the interest rate which will be used in our forward pricing of the underlying asset. This allows for a driftless diffusion process $dS_t = \sigma S_t dW_t$.

Game Component	**Interpretation in MARL Calibration**
Players	Each player corresponds to one Monte Carlo trajectory
Actions	The volatility chosen for the next timestep
Objective (Reward)	Shared reward based on how well simulated prices match market prices
Game Type	Cooperative multi-agent game

Table 1: MARL formulation of calibration as a cooperative multi-agent game

2 Reinforcement Learning Framework

The objective of RL is to determine an optimal policy π_θ^* that maps a given state to an action that that maximizes a cumulative reward signal. While tabular RL methods relies on discrete state-action lookup tables, DRL employs neural networks as universal function approximators. This reduces the curse of dimensionality which can enhance the possibilities of DRL. In this context, θ represents the high-dimensional weights and biases that parameterize the policy, enabling the agent to generalize across continuous state and action spaces.

In the stochastic game there is a shared reward $r(x_t^i, a_t^i, x_{t+1}^i)$ based on the aggregate calibration error of the ensemble. To reduce the computational burden of modeling millions of interactions, we employ a shared policy π_θ across all agents. The objective is to identify the Nash Equilibrium π^*, such that:

$$v(\pi_i; \pi_{-i}^*) \leq v(\pi^*) \quad \forall i \in \{1, \ldots, n\} \tag{1}$$

where $v(\pi_i; \pi_{-i}^*)$ denotes the expected cumulative reward of player i when playing strategy π_i while the remaining players follow the shared equilibrium policy π^*.

2.1 State Space

The state x_t^i represents the information available to trajectory i at time t. In the local volatility setting, the state is defined as

$$x_t^i = \left(\ln S_t^i, \ \frac{t}{T} \right),$$

where $\ln S_t^i$ denotes the log of the forward price and t/T is normalized time. Vadori (2022) chooses log-forward-spots rather than raw forward spot prices because log-returns exhibit more stable statistical properties (positive values, geometric price movements). An advantage of utilizing DRL is the extension of the state space. The information available at t is no longer constrained and this can be particularly useful for path dependent exotics. A common technique is to use an encoder such as an LSTM to enhance the agents knowledge of their current state through a highly representative latent state variable. Our results utilize the local volatility state.

2.2 Action Space

In the standard single-agent RL framework, the environment is modeled as a MDP. At each discrete time step t, the agent selects an action a_t from the continuous action space $\mathcal{A}$. However in the stochastic game we extend this to vectors of actions, states and rewards given the player i. The player i selects an action corresponding to the volatility used over the next time increment:

$$a_t^i = \sigma_t^i \in [\sigma_{\min}, \sigma_{\max}],$$

where the bounds $[\sigma_{\min}, \sigma_{\max}]$ define a realistic range for annualized volatility. In implementation, the policy outputs $\log(\sigma)$ rather than σ directly to ensure positivity and a smoother optimization landscape.

2.3 Environment and Transition Dynamics

Given state x_t^i and action σ_t^i, each path evolves according to a discretization of the stochastic diffusion process using Euler–Maruyama approximation:

$$\ln S_{t+1}^i = \ln S_t^i - \frac{1}{2}(\sigma_t^i)^2\,\delta + \sigma_t^i\sqrt{\delta}\,Z_t^i,$$

where δ is the time increment and $Z_t^i \sim \mathcal{N}(0,1)$ are independent standard normal random variables.

The removal of growth rates, and dividends allows for a simpler price dynamic for the agent to interpret as we move from one state to the next. While we do have a model for the environment, this transition is noisy due to the standard normal random variables.

2.4 Reward

The reward structure in RL is often scrutinized particularly in the MDP structure. Two major issues sparse rewards and credit assignment misalignment which can plague any RL approach. This is particularly profound in the MARL for calibration of options. The reward function measures calibration performance, quantifying how closely model-generated implied volatilies (IV's) match observed market IV's. The rewards are sparse as the model must wait for the end of the Monte Carlo Simulation to finish before calculating our pay off - this results in a single reward for many decisions. A common approach to overcome this is temporal differencing where we attribute a reward for each decision. Another problem is credit assignment, rewards can come at different frequencies and may be a latent variable which can lead to difficulty in attributing a specific decision as a good or a bad decision. This is clear in the MARL calibration problem as you have interacting agents which have no strong understanding of how their single decision will impact the success of all other trajectories.

3 MARL Calibration Rollout Phase

3.1 The Neural Network: Actor-Critic

The shared policy π_θ is implemented as a neural network with two output heads, following a standard actor-critic architecture. The network takes as input the state $x_t^i = (\ln S_t^i,\, t/T)$ and processes it through a shared trunk of fully connected layers with tanh activations. The resulting representation is then fed into two separate heads: the actor and the critic. Although tanh activation is more computationally expensive than ReLU, Vadori (2022) supports this decision through smoother and more efficient learning.

3.1.1 Actor Head

The actor outputs the parameters of a Gaussian distribution over $\log(\sigma) : \mu_\theta(x)$ and $\log\sigma_\theta(x)$, where $\mu_\theta(x)$ is the mean and $\log\sigma_\theta(x)$ is the log standard deviation.

An action is sampled from this distribution and then exponentiated to produce the volatility:

$$a = \mu_\theta(x) + \sigma_\theta(x)Z^\pi,\ Z^\pi \sim N(0,1) \qquad \sigma = \exp(a)$$

This random noise in the action space ensures appropriate exploration of policy parameters are explored. Policy gradient methods such as the one described align well with neural networks as we interact with the policy (probability distribution) directly through stochastic gradient descent compared to Deep Q-Network models which requires ϵ-greedy algorithms in order to explore the parameter space.

3.1.2 Critic Head

The critic outputs a scalar value function $V_\psi(x_t^i)$, which estimates the expected future reward from state x_t^i. The value function is crucial in computing advantage estimates later in the algorithm, reducing the variance of gradient updates and improving training stability.

3.2 Price Dynamics

At each timestep t, the algorithm evolves all n simulated stock price paths forward by one increment. This requires distinguishing between two separate sources of randomness, which play fundamentally different roles in the system.

3.2.1 Market Noise

Market noise represents the intrinsic randomness of asset price movements and enters directly into the diffusion process. In the state transition

$$\ln S_{t+1}^i = \ln S_t^i - \frac{1}{2}(\sigma_t^i)^2\,\delta + \sigma_t^i\sqrt{\delta}\,Z_t^i,$$

the market noise $Z_t^i \sim \mathcal{N}(0,1)$ is independently sampled for each trajectory at every timestep. This source of randomness is essential for generating realistic price paths and is present regardless of whether learning or exploration occurs.

3.2.2 Exploration Noise

Exploration noise governs the stochasticity of the policy and determines how actions deviate from their expected values. In the action sampling equation

$$a_t^i = \mu_\theta(x_t^i) + Z_t^{\pi,i}\,\sigma_\theta(x_t^i),$$

the exploration noise $Z_t^{\pi,i} \sim \mathcal{N}(0,1)$ is sampled only for the basis players, whose interpolation is a foundational aspect of the algorithm discussed later. Unlike market noise, exploration noise is part of the learning mechanism. It enables the policy to explore different volatility choices and is explicitly accounted for in gradient-based updates.

4 MARL Calibration Learning Phase

In DRL the reward is used as a signal to update the policy (neural network's weights and biases) to better approximate a high rewarding general state-action reward function. As discussed this is a noisy and difficult environment to discriminant signal from noise. This section compares Vadori's implied volaility reward to our pricing reward difference. The terminal reward is calculated by looking at the intrinsic value of the possible paths relative to given strikes in a set time til expiry:

$$\text{MC\_Price}(t,K) = \frac{1}{n}\sum_{i=1}^{n}\max(S_T^i - K, 0) \tag{2}$$

4.1 Approach 1: Implied Volatility Reward (The Vadori Method)

In the standard formulation proposed by Vadori (2022), the reward is calculated by directly comparing model-implied volatilities to market-implied volatilities. The reward function is defined as:

$$r(x_t, a_t, x_{t+1}) := -\sum_{m=1}^{n_t^o}\omega_t^m\left(\text{BS}^{-1}\left(\frac{1}{n}\sum_{j=1}^{n}\mathcal{X}_{t+1}^{j,m}\right) - \sigma_{mkt,t}^m\right)^2 \tag{3}$$

This approach requires the inversion of the Black-Scholes formula (BS^{-1}). A significant numerical challenge arises with deep Out-of-the-Money (OTM) options; if the simulated price is effectively zero, the inversion is undefined as no implied volatility exists. While adding a small epsilon (ϵ) prevents code execution failure, it introduces artificial noise into the reward signal, which can skew the optimization and hinder the model's ability to accurately fit the wings of the volatility surface.

4.2 Approach 2: Price-Based Reward (Proposed Method)

To prevent the instabilities inherent in IV inversion, we perform the loss calculation in the price domain. The reward is defined as:

$$r(x_t, a_t, x_{t+1}) := -\sum_{m=1}^{n_t^o} \omega_t^m \left(\hat{\mathcal{X}}_{t+1}^m - c_t^m\right)^2 \quad \text{where} \quad \hat{\mathcal{X}}_{t+1}^m = \frac{1}{n}\sum_{j=1}^{n} \mathcal{X}_{t+1}^{j,m} \tag{4}$$

By differencing in price space ($\hat{\mathcal{X}} - c$), the agent avoids these numerical stability issues associated with IV inversion. Empirically, this results in a smoother optimization landscape and a notable improvement in calibration results compared to the IV-space approach.

4.3 Basis Player Approximation

The standard policy gradient update requires computing the gradient contribution of every path $i \in N$ at each time step t:

$$g_\theta = \frac{1}{|N|}\sum_{i\in N}\sum_t \nabla_\theta \log \pi_\theta(a_t^i \mid x_t^i)\, A_\psi(x_t^i, a_t^i).$$

Assuming we use $T = 51$ timesteps and ~ 100000 paths, this would mean millions of gradient computations for each iteration. On top of this, we would need to sample and track the exploration noise Z^π for each of these pairs, making the problem computationally unreasonable. To reduce computational cost, Vadori (2022) introduces a subset of trajectories as basis players, where exploration noise is sampled only for this subset and interpolated to the remaining trajectories.

Let $\mathcal{B}$ denote the set of basis players. The policy gradient is then approximated using only the basis players $i \in \mathcal{B}$:

$$g_\theta = \frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}}\sum_t \nabla_\theta \log \pi_\theta(a_t^i \mid x_t^i)\, A_\psi(x_t^i, a_t^i).$$

For any trajectory $j \notin \mathcal{B}$, the exploration noise is obtained via interpolation:

$$Z_{j,t}^\pi = \text{interp}\big(\{x_t^i, Z_{i,t}^\pi\}_{i\in\mathcal{B}}\big)\big|_{x_t^j}$$

In implementation, we use k-nearest-neighbor interpolation, with $k = 5$. This process is done by finding the k basis players whose current log-spot is closest to trajectory j and taking an inverse-distance-weighted average of their exploration values Z^π. By using interpolation, we reduce the learning overhead by approximately a factor of $\frac{N}{\mathcal{B}}$. Since nearby trajectories in state space receive similar noise, there is a minimal effect on the quality of the gradient update.

4.4 Policy Optimization (PPO)

After collecting data for one pass through T timesteps, we use the popular policy gradient method Proximal Policy Optimization (PPO) to update the network. The gradient update for policy θ is defined as

$$g_\theta = E[\nabla_\theta \log \pi_\theta(a|x) A(x, a)],$$

where A is the advantage function discussed in the Advantage Estimation section. For now, it is sufficient to understand that when $A(x, a) > 0$, the policy gradient makes action a more probable, and if $A(x, a) < 0$, a becomes less probable.

A possible issue with the gradient update is its lack of bounds. If an unusually large advantage estimate results in a large gradient step, the new policy can change dramatically for the worse and destabilize training, sometimes referred to as gradient exploding. PPO can constrain the policy update by clipping the probability ratio, defined as

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid x_t)}{\pi_{\theta_{\text{old}}}(a_t \mid x_t)}.$$

The probability ratio can be interpreted as the factor increase in the likelihood of the new policy taking action a compared to the old policy. The PPO clips this ratio in the objective:

$$L_{\text{CLIP}}(\theta) = E\Big[\min\big(r_t(\theta)A_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t\big)\Big],$$

where ϵ is the clipping parameter. In the experiment, Vadori (2022) uses $\epsilon = 0.3$, allowing for flexibility of policy update but preventing any single iteration from changing the policy too drastically.

The full loss function is

$$L(\theta) = L_{\text{CLIP}}(\theta) - c_1(V_\psi(x_t) - R_t)^2 + c_2\, H[\pi_\theta(\cdot \mid x_t)],$$

where the terms correspond to policy improvement, value function accuracy, and entropy regularization. In the value loss component, the critic V_ψ is trained to match empirical returns $R_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$. The entropy bonus $H(\pi) = -E[\log \pi(a|x)]$ encourages exploration, preventing the policy from collapsing to a deterministic value too early in the training.

In practice, we use Minibatch Stochastic Gradient Descent, sampling from random minibatches for multiple passes over the data. This method is proven to extract more learning signal than one gradient step without using more data.

4.5 Advantage Estimation

The advantage function $A(x, a)$ measures the relative quality of an action compared to the expected value of the state:

$$A(x, a) = Q(x, a) - V(x),$$

where $Q(x, a)$ is the state-action value function and $V(x)$ is the critic function. By centering returns around the value function, the advantage formulation significantly reduces variance in gradient estimates compared to using raw returns. In practice, the state-action value function $Q(x, a)$ is not directly available, so the advantage must be approximated. Generalized Advantage Estimation (GAE) provides an efficient and flexible estimator that blends low-variance, biased estimates and high-variance, unbiased ones.

The Temporal-Difference (TD) residual measures the difference between the observed reward with discounted future value estimate and the current value estimate

$$\delta_t = r_t + \gamma V_\psi(x_{t+1}) - V_\psi(x_t).$$

GAE then constructs the advantage as a weighted sum of TD residuals:

$$A_t^{\text{GAE}} = \sum_{l=0}^{T-t} (\gamma\lambda)^l\, \delta_{t+l},$$

where $\lambda \in [0, 1]$ controls the bias-variance tradeoff. Notice that when $\lambda = 0$, GAE reduces to one-step TD estimation, yielding low variance but higher bias due to reliance on the critic. When $\lambda = 1$, GAE recovers a Monte Carlo estimate of the full return, which is unbiased but exhibits high variance.

In implementation, $\gamma = 1$ is used for no discounting, while $\lambda \approx 0.95$ is chosen to account for the need to look far ahead with sparse rewards.

The algorithm below summarizes our implementation of the approach in Vadori (2022).

Algorithm: MARLVol

Input: learning rate α, parameters (θ, ψ), basis size n_p
Initialize: market IV targets, Actor–Critic network
Repeat until convergence:

1. **Rollout Phase**
   (a) Initialize $\ln S_0^i = 0$ for all i
   (b) For $t = 0, \ldots, T-1$:
      i. Sample n_p basis players
      ii. For basis players, sample actions $a_t^i = \mu_\theta(x_t^i) + Z_{i,t}^\pi\, \sigma_\theta(x_t^i)$
      iii. Interpolate Z^π to all paths

iv. For all i, exponentiate action and update paths $\ln S^i_{t+1} = \ln S^i_t - \frac{1}{2}{\sigma^i_t}^2\delta + \sigma^i_t\sqrt{\delta}\, Z^i_t$

v. Compute reward r_t and store $(x^i_t, a^i_t, \log\pi_\theta, V_\psi, r_t)$ for basis players

2. **Learning Phase**

   (a) Compute GAE advantages A^{GAE}_t and normalize

   (b) For multiple epochs:

   - Sample minibatches
   - Compute PPO loss
   - Update θ, ψ via gradient ascent

Output: trained policy θ

5 Experiment

5.1 Market Data

The model was calibrated using SPX call option market data from May 5th, 2025 at 9:31 AM. The SPX option data consists of two anchor maturities at 35 and 53 calendar days, with three additional maturities at 39, 44, and 49 days interpolated evenly between the anchors for evaluation. Training is performed exclusively on the two anchor maturities; the interpolated maturities serve as out-of-sample evaluation to assess the generalization of the calibrated volatility surface across the term structure. Strikes are parameterized as moneyness relative to the forward price. For each maturity, the dataset contained strike prices, forward implied volatilities, option mid prices, forward-adjusted option prices, vegas, and forward prices.

For each maturity, the strike closest to the forward price was used to determine the at-the-money (ATM) implied volatility. This ATM implied volatility was then used to construct a Black–Scholes baseline model with a flat volatility smile. The MARL model was trained to improve upon this baseline by minimizing weighted pricing error across strikes.

The reward weighting scheme used vega weighting:

$$w_i = \frac{\text{Vega}_i}{\frac{1}{N}\sum_{j=1}^{N}\text{Vega}_j},$$

which places greater emphasis on strikes with higher sensitivity to volatility changes. This weighting was chosen because pricing errors near high-vega regions have a larger impact on the resulting implied volatility surface.

5.2 Policy Network Architecture

A shared PPO actor–critic network was trained jointly across both maturities. The network consisted of three fully connected hidden layers with 50 neurons per layer and tanh activation functions. The actor output mean was constrained to a bounded range corresponding to realistic volatility levels, while the exploration variance was parameterized through a learnable log-standard deviation. In Vadori (2022), the variance was a second head in the actor output, but we chose a learnable parameter instead to avoid the exploration spiking that was collapsing the learning.

5.3 Calibration Environment

We used $N = 50000$ Monte Carlo paths for the agents, in which $B = 100$ were basis players. To reduce variance among path rollouts, we used 10 parallel rollouts for each iteration by following the Vadori (2022) procedure. The simulation timestep was set to $dt = \frac{1}{252}$, corresponding to daily discretization.

5.4 Training Parameters

Training was performed jointly across both maturities using PPO with generalized advantage estimation (GAE). The primary hyperparameters used in training are shown below.

Parameter	Value
Epochs	350
PPO Epochs per Update	5
Learning Rate	2×10^{-4}
PPO Clip Parameter	0.1
Discount Factor γ	1.0
GAE Parameter λ	0.997
Entropy Coefficient	0.005
Value Loss Coefficient	0.05
Gradient Clip Norm	5.0

Early stopping was implemented with a patience of 50 epochs after an initial warmup period of 75 epochs. We included gradient clipping to avoid large updates and increased the GAE parameter to introduce more variance in the training.

5.5 Results

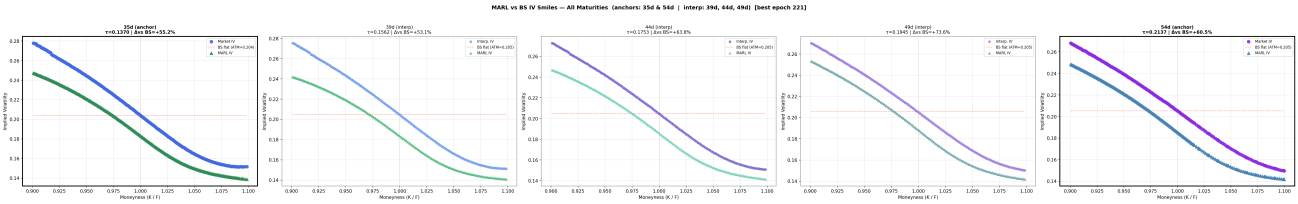

Figure 1: Implied Volatility Smiles.

Maturity	MARL Error	Black–Scholes Error	Improvement
35d	46629.081888	104013.445169	+55.2%
39d	39477.371360	84187.354767	+53.1%
44d	34386.680123	95009.279436	+63.8%
49d	27652.021557	104623.119295	+73.6%
54d	54012.125788	136885.741928	+60.5%

Table 2: Calibration error comparison between the MARL model and Black–Scholes benchmark.

5.6 Design Choices

The biggest design change to the original paper is the use of market prices as reward function rather than implied volatility. This saw results improve massively as numerical stability issues in implied vol calculations were no longer skewing the reward function.

The vega, inverse vega, and uniform weighting were evaluated and ultimately the vega weight came out to be the most promising. By putting the largest concentration on the forward price ATM options it encourages the curve to be learnt rather than shifting the implied volatility to the deep ITM and deep OTM options.

6 Discussion

6.1 Computational Requirements

Without the novelty of using basis players the opportunity to utilize such a high number of trajectories / agents would become computationally infeasible. By relaxing gradients to a smaller subset and keeping

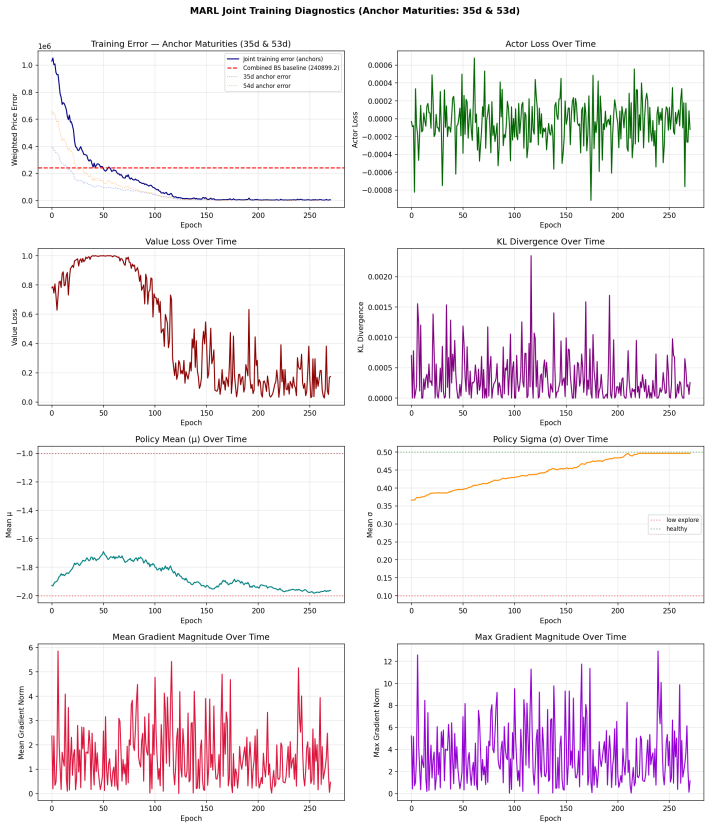

Figure 2: MARL Calibration Training Diagnostics.

rewards sparse the computational loads are massively reduced and it becomes feasible to run on CPUs within 20-30 minutes.

6.2 Dupire Local Volatility Representation

The MARL agent restricts its state space to match that of the Dupire local volatility model. The traditional method requires rich greek data which may be difficult to find in lower liquidity markets. In particular we need to solve the following PDE:

$$\sigma_{loc}^2(K,T) = \frac{\frac{\partial C}{\partial T} + (r-q)K\frac{\partial C}{\partial K} + qC}{\frac{1}{2}K^2\frac{\partial^2 C}{\partial K^2}} \tag{5}$$

The second-order derivative with respect to strike ($\frac{\partial^2 C}{\partial K^2}$) is highly sensitive to price noise and requires a dense, differentiable surface of option prices. Calculating these Greeks often leads to numerical instability or non-physical (negative) volatilities.

The MARL agent overcomes these limitations by mapping the local volatility surface through simulation rather than differentiation. By utilizing a Deep Reinforcement Learning framework, the agent learns the policy $\pi(a_t|x_t)$, where the action a_t corresponds to the local volatility $\sigma(S_t, t)$.

6.3 Forward Price Initiative

With the assumption of $r = 0$ in the stochastic diffusion process we isolate the paths as the only variable to fix. Failure to incorporate the time value of money though would result in a skewed result which would place paths higher rather than the negative skew which is common in market prices of options (as a result of stylized financial time series). This isolates the issue to volatility movement alone and not incorporating over drift measures which is not the intention of the project. However, we believe that the downwards shift in the MARL calibration fit to the market IV smile may be a result of noisy regression when calculating the forward prices.

7 References

Nelson Vadori. Calibration of derivative pricing models: a multi-agent reinforcement learning perspective. *arXiv preprint arXiv:2203.06865*, 2022. URL https://arxiv.org/abs/2203.06865.